

Jack Wu

Backend Engineer | Wu Cheng-Ju

hotrabbit1016@gmail.com | +886 955 684 107 | hotrabbit1016.github.io/resume

PROFESSIONAL SUMMARY

Backend Engineer with five years of experience building production systems for online entertainment platforms. Primary stack: C#/ .NET, with Go for audit data pipelines and Python for physical track control and computer vision. Work spans transaction services, real-time delivery, distributed coordination, hardware integration, and AI inference. I design for failure paths such as duplicate or missing signals, latency, race conditions, unreliable inference, and device faults.

EXPERIENCE

Sentinel IT | Backend Engineer | Jan 2024 - Present

- Moved with the original team and product line after a change of ownership. Own work across the table-client rewrite, operations back-office, chat-platform audit logging, physical track control, and AI result judging.
- Built production services in .NET, Go, and Python, covering multi-pod coordination, asynchronous audit pipelines, hardware abstraction, and real-time computer vision.
- Remotely led one developer based in the Philippines for about a year, covering requirement breakdown, technical direction, code review, and delivery tracking.

Ifalo International / Dingshun Digital | Backend Engineer | Apr 2021 - Dec 2023

- Worked continuously with the same owner, team, and product line across two companies. Progressed from report conversion and table-state handling to player betting, cancellation, and settlement services.
- Implemented transactional betting flows, pushed settlement results over SignalR, and used RabbitMQ events for cross-service state changes. Maintained the production service for about four years.
- Developed the on-site WPF table console; wrote more than 90% of one generation's main view and core logic, including double confirmation, layered error handling, and full-state diagnostics.

TECHNICAL SKILLS

Backend: C# / .NET 10, ASP.NET Core, EF Core, Go (Gin, Ent, Uber Fx), Python

Realtime & Messaging: SignalR, WebSocket, RabbitMQ, Redis Streams

Data Stores: MySQL, Redis (Lua, distributed locks), ClickHouse, Elasticsearch

Infrastructure: Docker, Kubernetes, Helm, GitLab CI, ArgoCD, OpenTelemetry

AI & Vision: YOLO, Roboflow, PyTorch, TensorFlow, OpenCV, ONNX

Hardware: SPI (MCP23S17), UART / RF, Raspberry Pi, basic soldering and PCB design discussions

EDUCATION & LANGUAGES

National Chung Hsing University, B.S. in Electrical Engineering | Mandarin (native) | English (TOEIC 770)

SELECTED PROJECTS

Table Client Rewrite | 2025 - 2026

.NET 10 | Redis | Kubernetes | xUnit

- Rebuilt a single-machine WPF application as a multi-pod service. Independently designed and implemented the main service and its tests.
- Implemented table leases with Redis and Lua so acquire, renew, and release are atomic, preventing two pods from taking the same table and processing its signals twice.
- Used supervisor-worker scheduling with one DI scope per table. A failed renewal stops its worker; state-pattern and TimeProvider tests cover eight table states and the scheduling core.
- Removed plaintext credentials and connection strings from version control; Kubernetes Secrets now inject them at deployment time.

Chat Platform Audit Logging | 2026

Go | Gin | Redis Streams | ClickHouse | Uber Fx

- Delivered the system end to end, from evaluating options and writing the design to production release, so auditors can answer who changed what and when.
- Sent records through a Redis stream and consumer group before ClickHouse, keeping audit work off the request path; used daily partitions and TTL-based retention.
- Collected HTTP meaning in middleware and before/after snapshots in services, then merged both paths. Replaced raw IDs and enums with names auditors can read directly.
- Used closed types with shared validation so invalid query values return 400. Kept the backend timezone-agnostic with a loose backstop while the frontend owns the precise boundary.

Physical Track Control & Firmware | 2024 - 2026

Python | SPI | UART / RF | Raspberry Pi | Flask | OpenTelemetry

- Sole developer of the control software and monitoring UI for eight physical marble tracks, covering sensors, gates, race flow, and OBS scene switching.
- Drove MCP23S17 I/O expanders over SPI to read up to 28 optical sensors, auto-detected two board generations, and provided a simulation fallback when hardware is absent.
- Layered retries, reinitialisation, circuit checks, and resource cleanup around lossy RF links so the system can recover from device faults without manual intervention.
- Isolated eight track-specific flows behind strategy and factory patterns. Three systemd services keep track control running even if the monitoring UI fails.

Marble Race Result Judging | 2024 - 2026

Python | YOLO | Roboflow | PyTorch | OpenCV | ONNX | FastAPI

- Built the production system independently: model training, judging logic, query API, and on-site interface. It replaced manual video review.
- Trained each ball as a separate class to avoid multi-object tracking ID switches, then modelled drop judging as a four-state machine.
- Expressed thresholds in time rather than frames. Two-stage confirmation, a survival threshold, and stale cleanup separately absorb missed detections, false positives, and ghost tracks.
- Ranked by the last detection time rather than confirmation time to preserve fairness, and ran inference on a separate thread to keep the on-site UI responsive.